

ST5003CMD

LINUX FUNDAMENTAL

Operating Systems, Security,
and Networks

BSc (Hons) Computer Science with AI Softwarica College of IT & E-commerce

STUDENT NAME		Ayusha Sapkota
STUDENT ID		250023
SUBJECT LEADER		Rupak Rajbanshi

Investigating Process Lifecycles and OS Interaction

1. Introduction

This laboratory exercise focuses on understanding how the operating system manages processes and how C programs interact with the operating system. The practical work demonstrates process execution, process identification, and exit status.

Three C programs were created and executed in a Linux environment. The first program was used to observe a process while it was running. The second program demonstrated how a process can obtain its own process ID and the ID of its parent process. The final program demonstrated how a program can return an exit status to the operating system.

2. Running and Monitoring a Process

The first part of the practical involved creating a C program that remains active for approximately 30 seconds. This allowed the running process to be observed from another terminal.

```
root@AYUSHA:~# nano task_alive.c
root@AYUSHA:~# gcc task1_alive.c -o task1
```

Figure 1: Creating the Source File and compiling the program

```
root@AYUSHA:~# ./task1 &
[1] 558
I am starting...
root@AYUSHA:~# I am finished.
```

Figure 2: compilation and execution of task1.

The **&** symbol allows the program to run in the background while the terminal remains available for other commands.

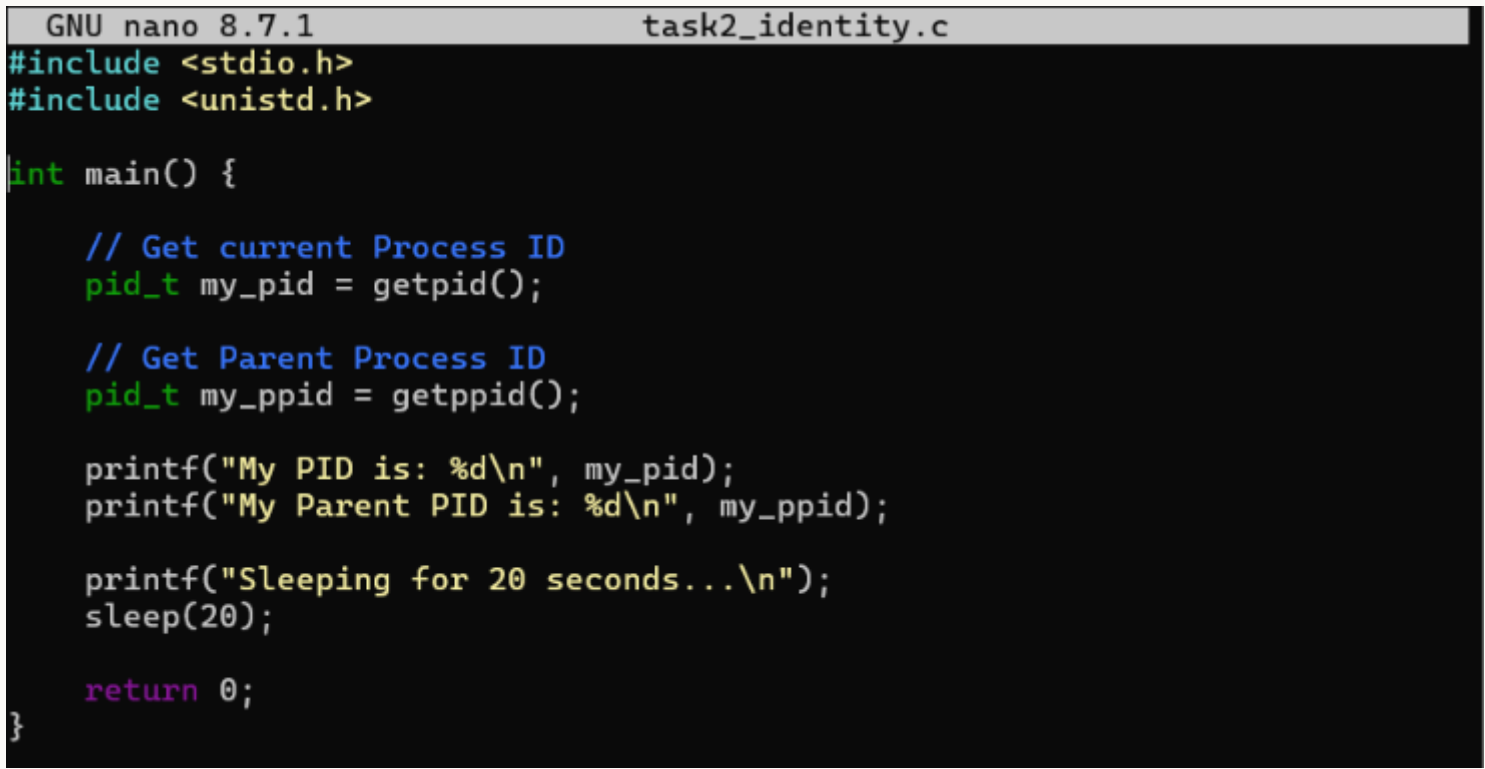
To monitor the process, a second terminal was opened while the program was running. This displayed information about the running process, including its Process ID (PID). After approximately 30 seconds, the program completed and displayed: I am finished.

```
root@AYUSHA:~# ps aux | grep task
root      571  0.0  0.0  4128  2404 pts/0    S+   07:26   0:00 grep --co
lor=auto task
```

Figure 3: ps aux | grep task1 and the running process.

3. Identifying the Process and Its Parent

The second part focused on identifying a process and its parent process. Linux assigns every running process a unique **Process ID (PID)**, while the **Parent Process ID (PPID)** identifies the process that created it. The `getpid()` function returns the PID of the current process, while `getppid()` returns the PID of its parent process. The following code was entered:



```
GNU nano 8.7.1 task2_identity.c
#include <stdio.h>
#include <unistd.h>

int main() {

    // Get current Process ID
    pid_t my_pid = getpid();

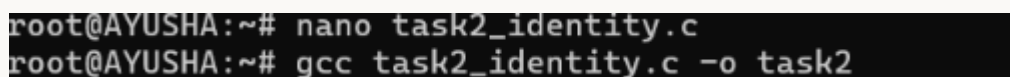
    // Get Parent Process ID
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);

    return 0;
}
```

Figure 4: Text Editor



```
root@AYUSHA:~# nano task2_identity.c
root@AYUSHA:~# gcc task2_identity.c -o task2
```

Figure 5: Creating the Source File and compiling the program

The program was then executed using:

```
root@AYUSHA:~# ./task2 &  
[1] 678  
My PID is: 678  
My Parent PID is: 494  
Sleeping for 20 seconds...
```

Figure 6 : display of the PID and PPID, task2.

To Verifying the Process Information, The PID displayed by the program was used with the following command:

```
root@AYUSHA:~# ps -p 678 -o pid,ppid,cmd  
    PID    PPID  CMD  
[1]+  Done                ./task2
```

Figure 7: verified process information.

Here, 678 represents the PID obtained from the program. The command displayed the PID, PPID, and command associated with the process, allowing the process information to be verified.

4. Exit Codes and OS Feedback

This part demonstrated how a C program communicates its execution status to the operating system through an exit code. An exit code of 0 generally indicates successful execution, while a non-zero value indicates a non-success status.

```
root@AYUSHA:~# nano task3_exit.c
root@AYUSHA:~# gcc task3_exit.c -o task3
```

Figure 8: Creating the Source File and compiling the program

The program accepts a number from the user. A non-negative value causes the program to return 0, while a negative value causes it to return 1. So here we check a Successful Execution and an Unsuccessful Execution.

```
root@AYUSHA:~# ./task3
Enter a number (positive for success, negative for fail): 5
Success!
root@AYUSHA:~# echo $?
0
root@AYUSHA:~# ./task3
Enter a number (positive for success, negative for fail): -5
Failure!
root@AYUSHA:~# echo $?
1
```

Figure 9: Terminal showing successful execution and unsuccessful execution

Conditional Execution and Termination

Programs often branch based on user input. The OS tracks the final exit code of a process to determine whether a calling script or automation pipeline should continue or halt. Every process, when it terminates, returns an integer status code to its parent. By convention, 0 means success and any non-zero value signals an error or an alternate outcome. `getpid()` is called first so the process announces its own identity before doing anything else useful for tracing which OS process handled which run.

```
root@AYUSHA:~# nano task5_control.c
root@AYUSHA:~# cat task5_control.c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int choice;

    printf("Current PID: %d\n", getpid());

    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing...\n");
        sleep(1);
        return 0;
    } else {
        printf("Exiting...\n");
        return 1;
    }
}
```

Figure 10. Creating and viewing task5_control.c

```
root@AYUSHA:~# gcc task5_control.c -o task5
root@AYUSHA:~# ./task5
Current PID: 1317
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
root@AYUSHA:~# echo $?
0
root@AYUSHA:~# ./task5
Current PID: 1336
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
root@AYUSHA:~# echo $?
1
```

Figure 11. execution of user selects Yes (exit code 0) and user selects No (exit code 1)

